

Dynamic Batch Formation Simulator

1. Problem statement

This project implements fixed, dynamic, and continuous batching for real LLM inference inside Docker. It compares output-token throughput, request latency, time to first token (TTFT), and active batch size under dense and sparse request arrivals.

2. Architecture

JSONL workload -> waiting queue -> scheduler -> Qwen inference -> CSV/JSON metrics -> median graphs

The benchmark starts a clean process for one strategy at a time. Runs remain sequential. The order rotates between repetitions to reduce order bias.

3. Batching strategies

- **Fixed batching** waits for four requests, then runs that closed batch until every request finishes. The final batch may be smaller.
- **Dynamic batching** starts a closed batch when four requests are ready or the oldest waiting request has waited 25 ms.
- **Continuous batching** checks for free slots after every token step and admits waiting requests as active requests finish.

4. Technology stack

- Python 3.11
- PyTorch 2.8.0 with CUDA 12.8
- Transformers 4.46.3
- Qwen/Qwen2.5-0.5B-Instruct in FP16
- Docker Compose with NVIDIA GPU access
- Matplotlib 3.9.2

5. Experimental setup

The final experiment ran on an NVIDIA GeForce RTX 5050 Laptop GPU with 8151 MiB of memory and driver 592.82, connected to AC power in High performance mode. Batch size was four and the dynamic wait limit was 25 ms. Each strategy ran three times per workload. Reported bars are medians; error bars span the minimum and maximum run. Raw results and device details are in `results/windows-rtx-5050-2026-10-10/`.

Every strategy received the same prompt sequence and per-request `max_new_tokens` values. Each run completed 30 unique requests and generated 960 output tokens. This check prevents missing, duplicated, or unequal work from affecting the comparison.

6. Workloads and parameters

Workload	Requests	Arrival interval	Output limits
Dense	30	20 ms	16, 32, and 48 tokens
Sparse	30	400 ms	16, 32, and 48 tokens

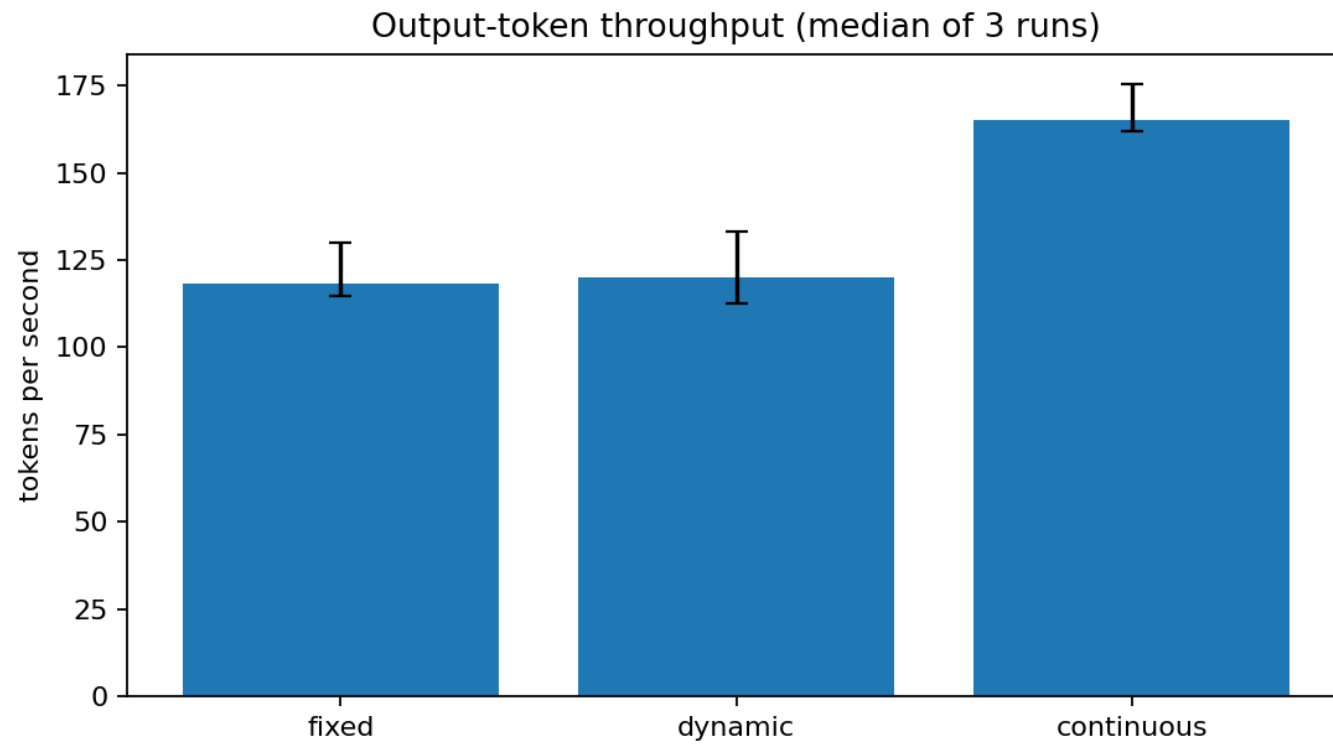
The dense workload keeps requests available for batching. The sparse workload exposes the queueing cost of waiting for a full fixed batch.

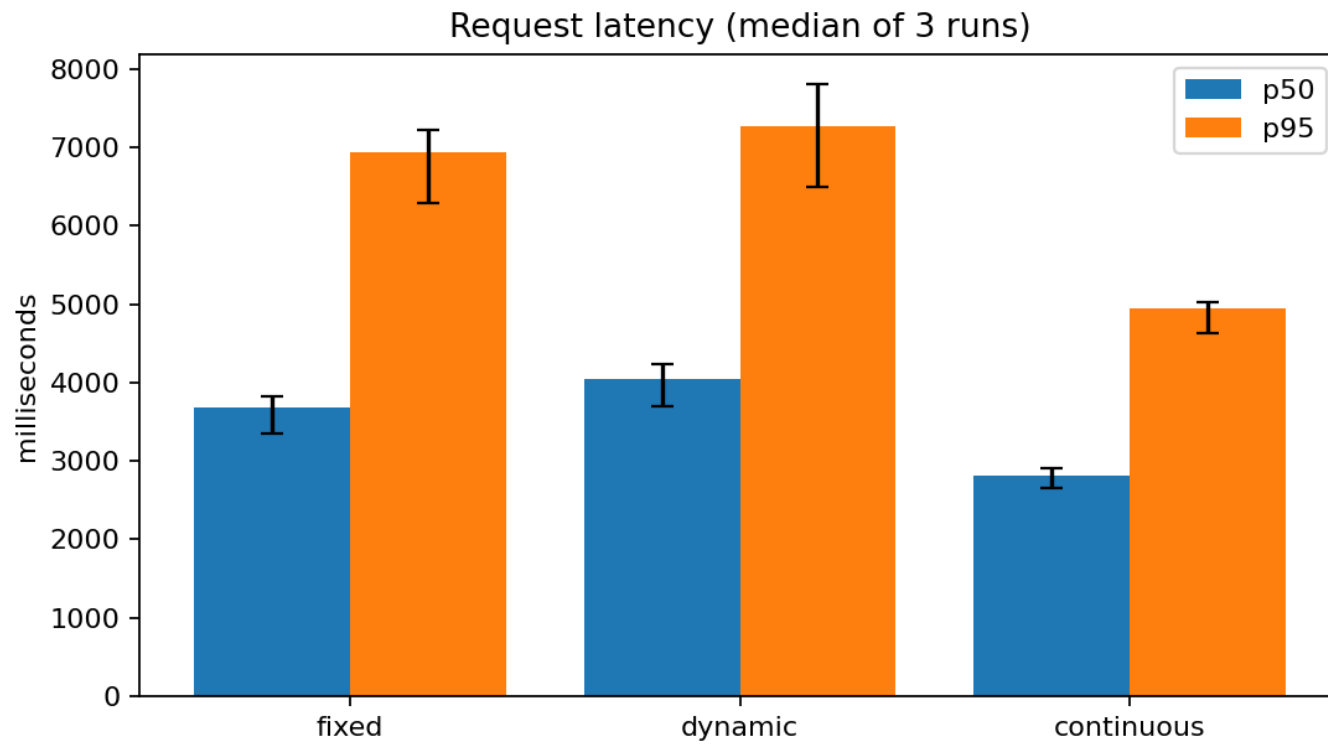
Latency and TTFT are measured from scheduled arrival to completion and first token, respectively, so both include queueing. Throughput divides all 960 output tokens by the time from the first scheduled arrival to the last completion; the sparse result therefore includes intentional gaps between arrivals. Lower latency and TTFT are better; higher throughput is better. No scheduler is guaranteed to win every metric.

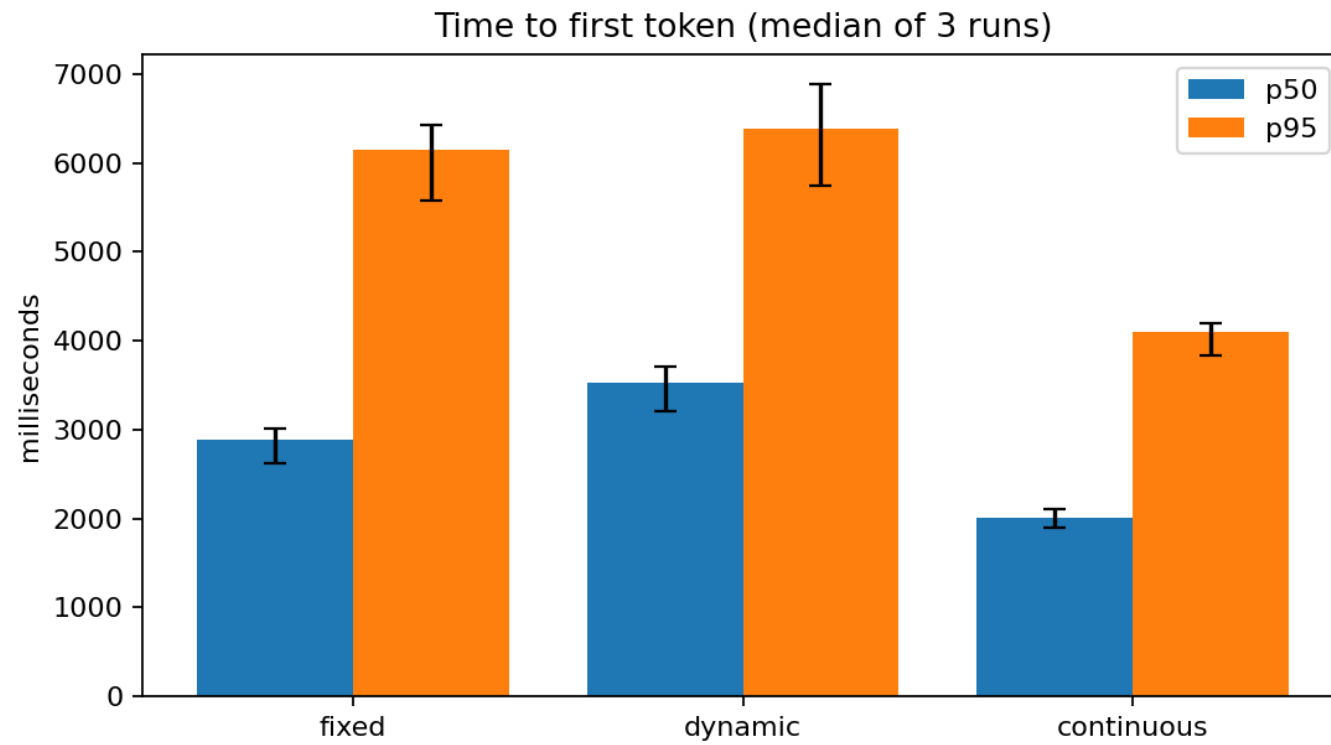
7. Results

Dense workload

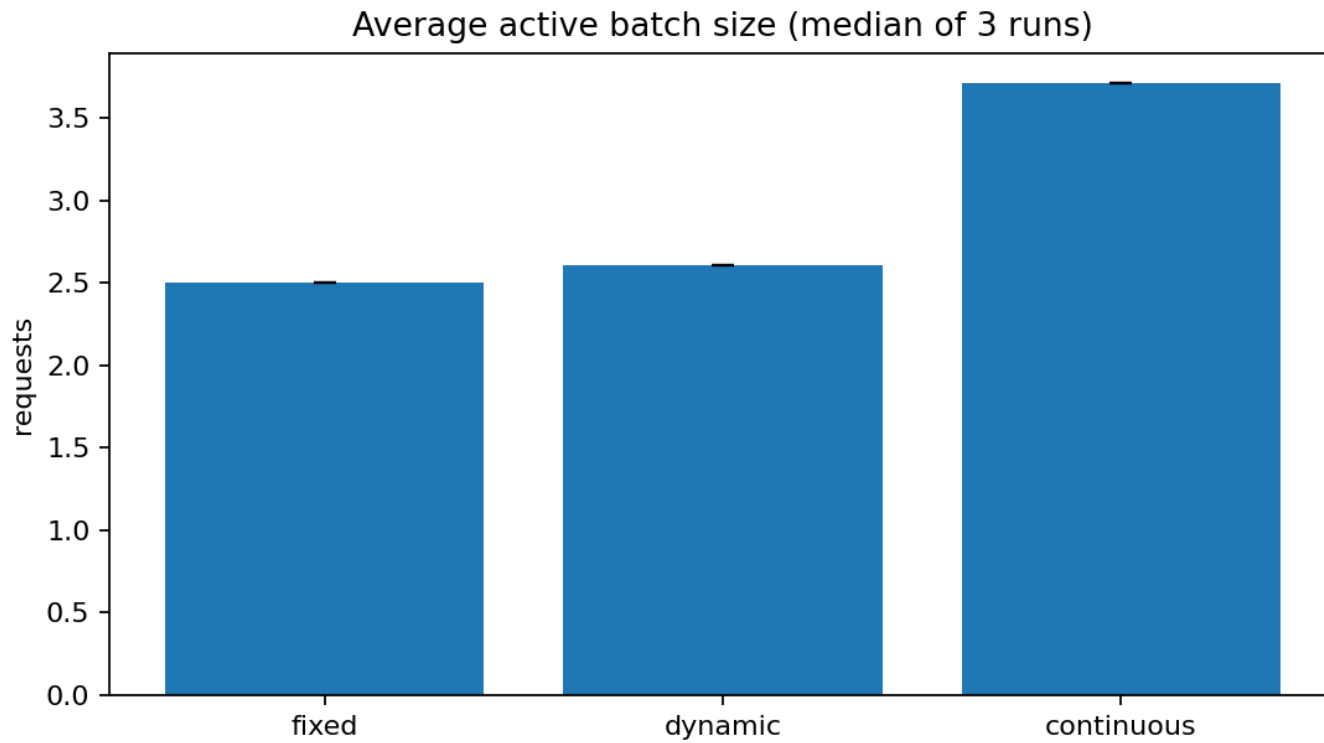
Strategy	Tokens/s	Latency p50 (ms)	Latency p95 (ms)	TTFT p50 (ms)	TTFT p95 (ms)	Avg. active batch
Fixed	118.32	3686.74	6932.36	2890.82	6155.18	2.50
Dynamic	120.09	4040.71	7265.09	3521.04	6383.40	2.61
Continuous	165.11	2815.03	4941.50	2014.44	4092.68	3.71







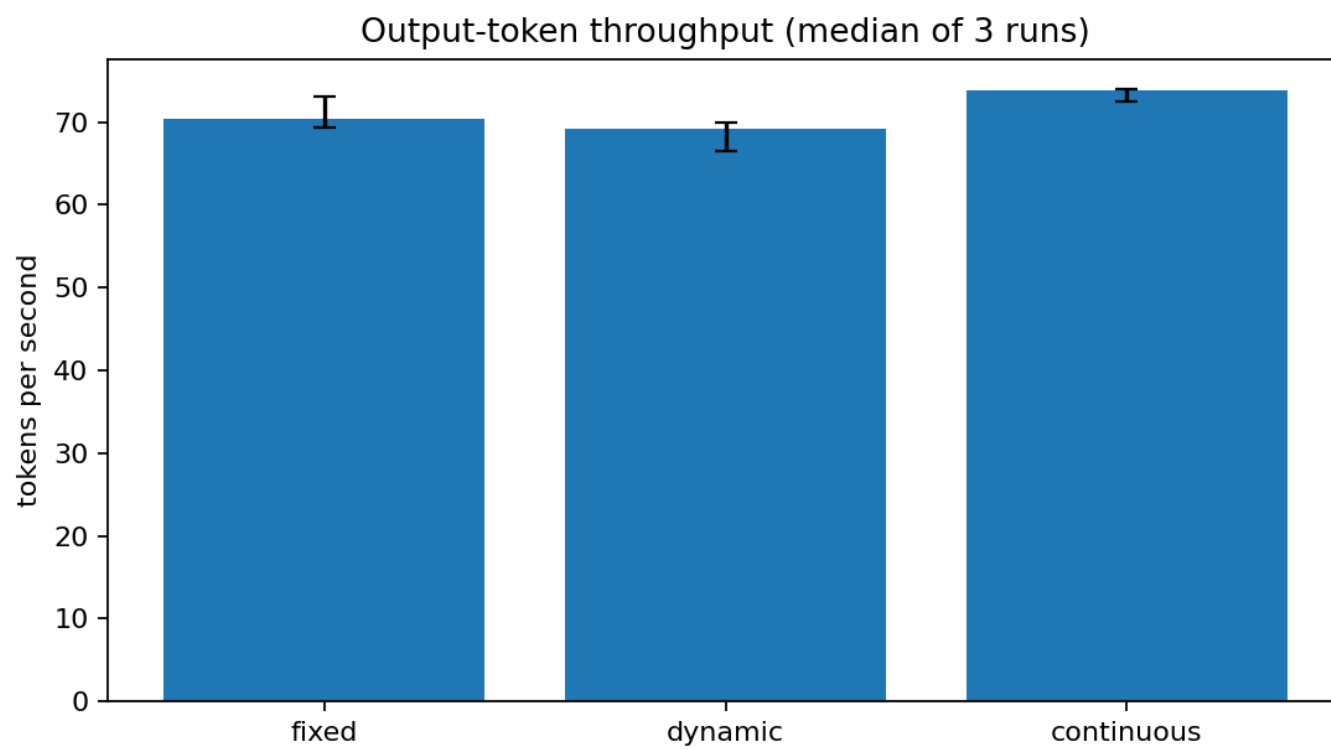
Dense average active batch size

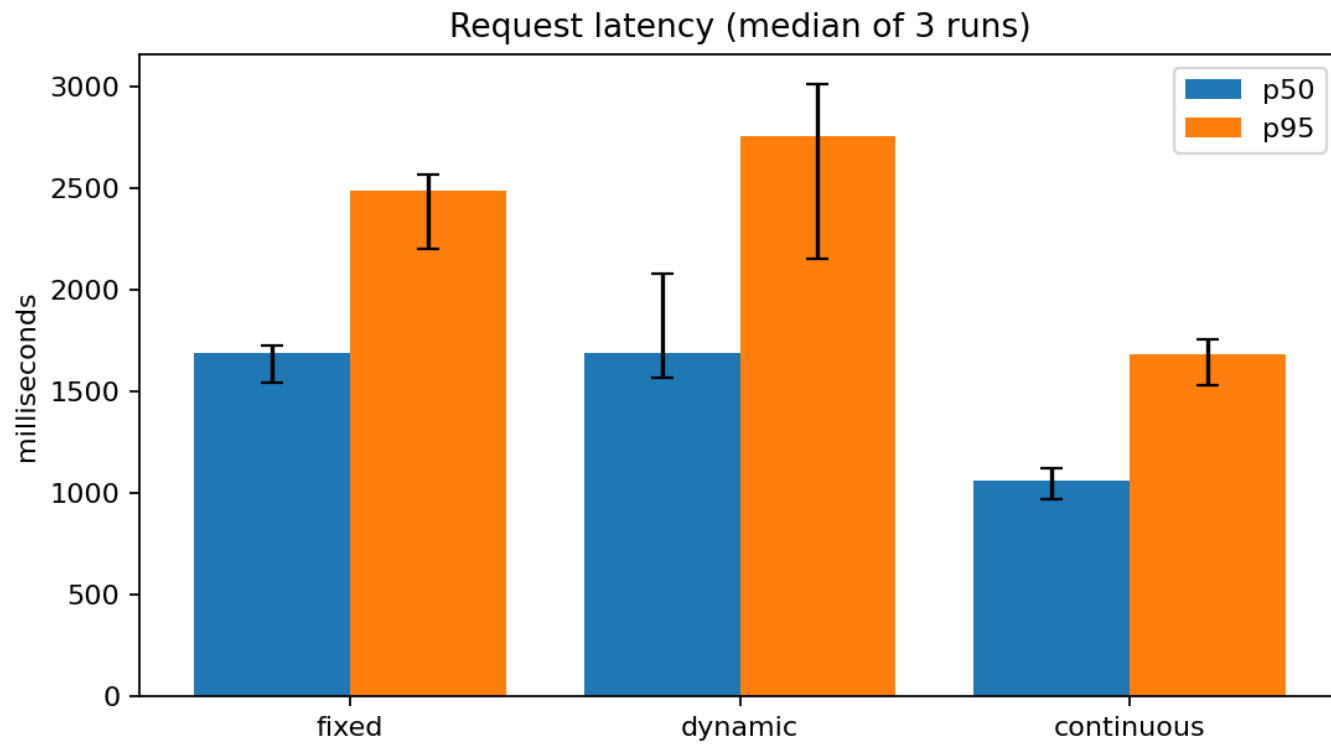


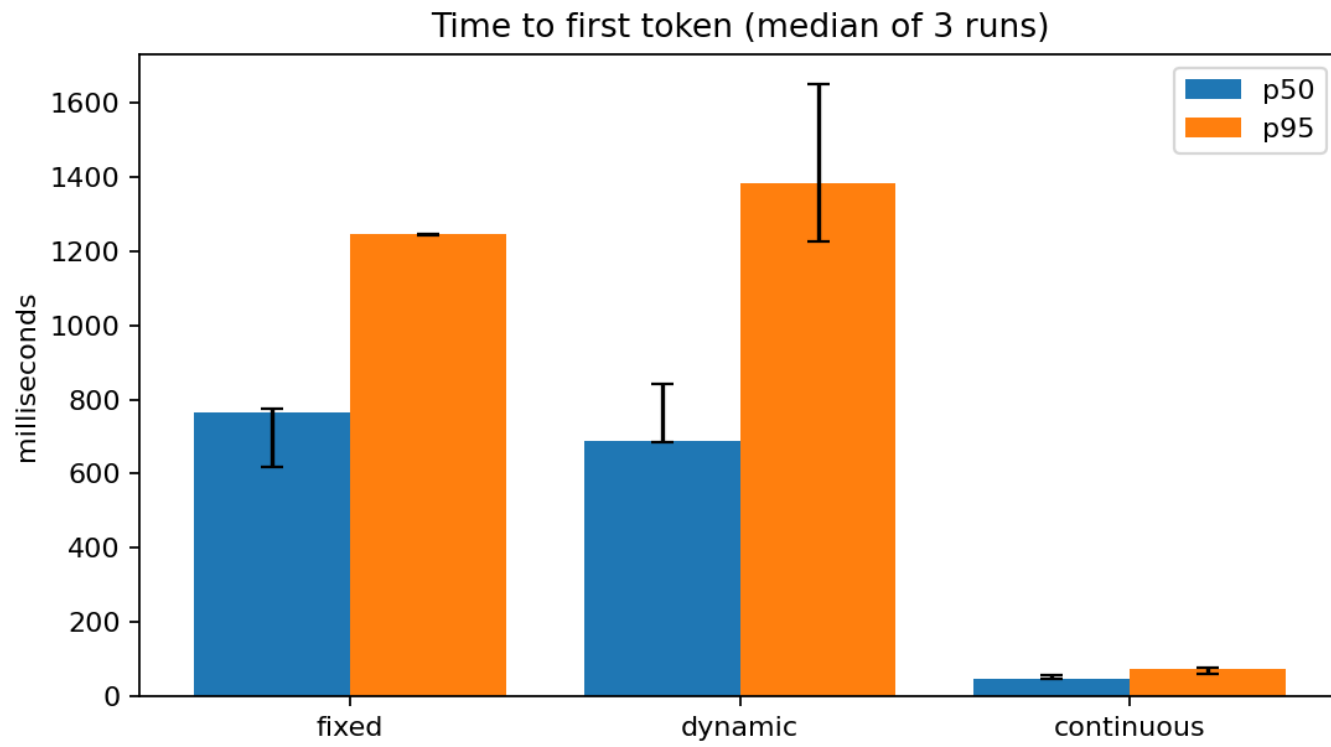
Continuous batching achieved the highest dense throughput and the lowest dense p50 and p95 latency. Its average active batch size of 3.71 shows that slot refilling kept more requests active. Dynamic throughput was close to fixed, but its median TTFT was higher. Its 25 ms deadline starts the first batch with only two requests, while fixed waits for four; because dynamic keeps that batch closed until completion, later requests queue behind it. In the first dense run, the first request's TTFT was 58 ms with dynamic versus 124 ms with fixed, but median queue wait rose to 3187 ms versus 2609 ms.

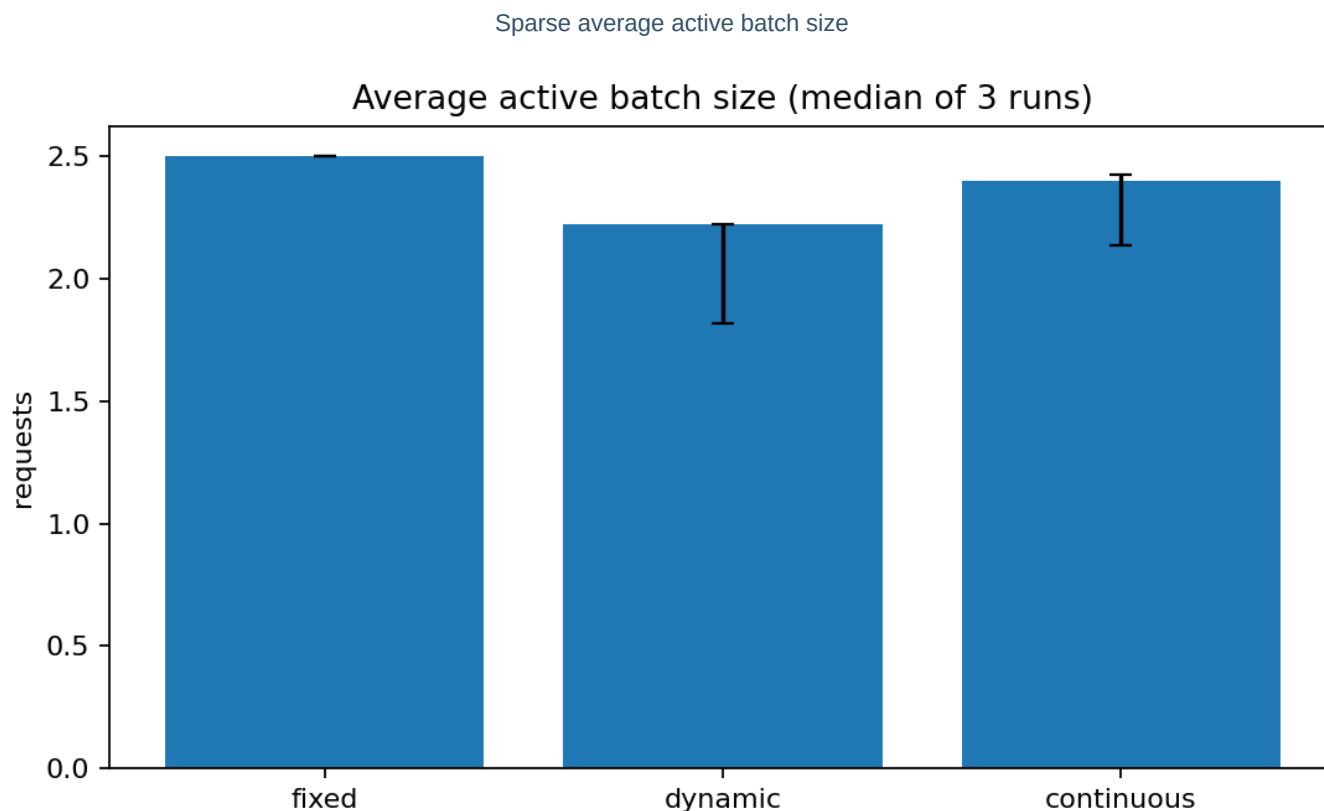
Sparse workload

Strategy	Tokens/s	Latency p50 (ms)	Latency p95 (ms)	TTFT p50 (ms)	TTFT p95 (ms)	Avg. active batch
Fixed	70.37	1686.95	2484.21	765.96	1245.59	2.50
Dynamic	69.19	1686.12	2752.52	687.55	1381.99	2.22
Continuous	73.80	1060.20	1680.94	46.25	73.07	2.40









Sparse throughput was similar because the arrival schedule dominated the makespan. Dynamic gave the first request a token after 51 ms, compared with 1250 ms for fixed, but its median TTFT improved by only 78 ms and its p95 TTFT and latency were worse. The closed, often undersized batches reduce its capacity to clear later arrivals. Continuous batching admitted requests into free slots and achieved the lowest latency and TTFT.

8. Limitations and conclusion

This version performs real Qwen inference, but it recomputes active sequences on every token step with `use_cache=False`. It does not implement a production KV-cache manager, paged attention, or multi-GPU execution. Results apply to this model, hardware, workload, and parameter set.

The experiment demonstrates the intended trade-off without forcing one ranking. Fixed batching can collect larger closed batches, dynamic batching limits the initial wait but can build a later queue, and continuous batching uses iteration-level admission to improve utilization and responsiveness. On this setup, continuous batching performed best overall. The 25 ms dynamic setting did not consistently outperform fixed batching; a different wait limit would require a new, separately labeled experiment rather than changing these recorded results.